Tcp全连接队列和tcpdump抓包

Tcp全连接队列：

> 在服务器来不及进行accept获取连接的时候，内核底层的tcp监听套接字允许用户继续三次握手，建立连接成功，但是不能建立成功的太多！即listen函数的第二个参数 **backlog** + 1的数量
>
> **连接本质是内核中的一种数据结构/结构体对象！**
>
> 注意！并不代表服务端只能同时处理**backlog** + 1个连接！
>
> **全连接队列中链入的是来不及处理的客户端连接！！！！！**
>
> ---
>
> 全连接队列本质就是**生产者--消费者模型，该队列不能为空！也不能太长！**
>
> 为什么不要太长？因为队列长了一定意味着队列尾部的等待时间一定长，浪费空间又浪费时间！

内核层理解socket：

创建监听套接字：

```
struct task_struct {
/* open file information */
    struct files_struct *files;
```

```
struct files_struct {
  /*
   * read mostly part
   */
    atomic_t count;
    struct fdtable *fdt;
    struct fdtable fdtab;
  /*
   * written part on a separate cache line in SMP
   */
    spinlock_t file_lock ____cacheline_aligned_in_smp;
    int next_fd;
    struct embedded_fd_set close_on_exec_init;
    struct embedded_fd_set open_fds_init;
    struct file * fd_array[NR_OPEN_DEFAULT];
};
```

```
struct file {
    /*
     * fu_list becomes invalid after file_free is called and queued via
     * fu_rcuhead for RCU freeing
     */
#endif
    /* needed for tty driver, and maybe others */
    void            *private_data;
```

BSD--通用socket接口

```
struct socket {
    socket_state        state;
    unsigned long       flags;
    const struct proto_ops  *ops;
    struct fasync_struct    *fasync_list;
    struct file         *file;
    struct sock         *sk;
    wait_queue_head_t   wait;
    short               type;
};
```

这里的sk 指针本质上是c语言风格的多态技术！

问题：socket是如何得知自己的sk 成员指向的是 tcp_sock还是 udp_sock呢?

```
int socket(int domain, int type, int protocol);
```

套接字方法集

```
struct proto_ops {
    int     family;
    struct module   *owner;
    int     (*release)   (struct socket *sock);
    int     (*bind)      (struct socket *sock,
                          struct sockaddr *myaddr,
                          int sockaddr_len);
    int     (*connect)   (struct socket *sock,
                          struct sockaddr *vaddr,
                          int sockaddr_len, int flags);
                          ...
}
```

```
struct udp_sock {
    /* inet_sock has to be the first member */
    struct inet_sock inet;
    ...
}
```

```
struct tcp_sock {
    /* inet_connection_sock has to be the first member of tcp_sock */
    struct inet_connection_sock inet_conn;
```

```
struct inet_connection_sock {
    /* inet_sock has to be the first member! */
    struct inet_sock        icsk_inet;
    struct request_sock_queue icsk_accept_queue;
```

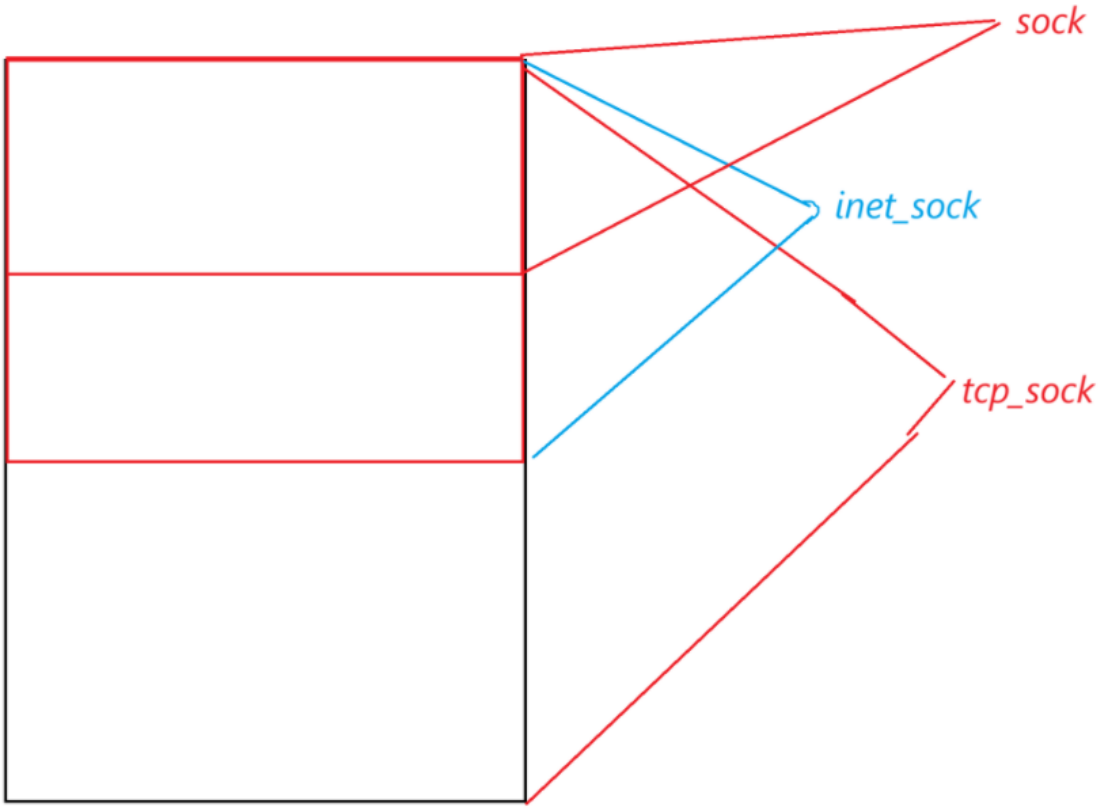
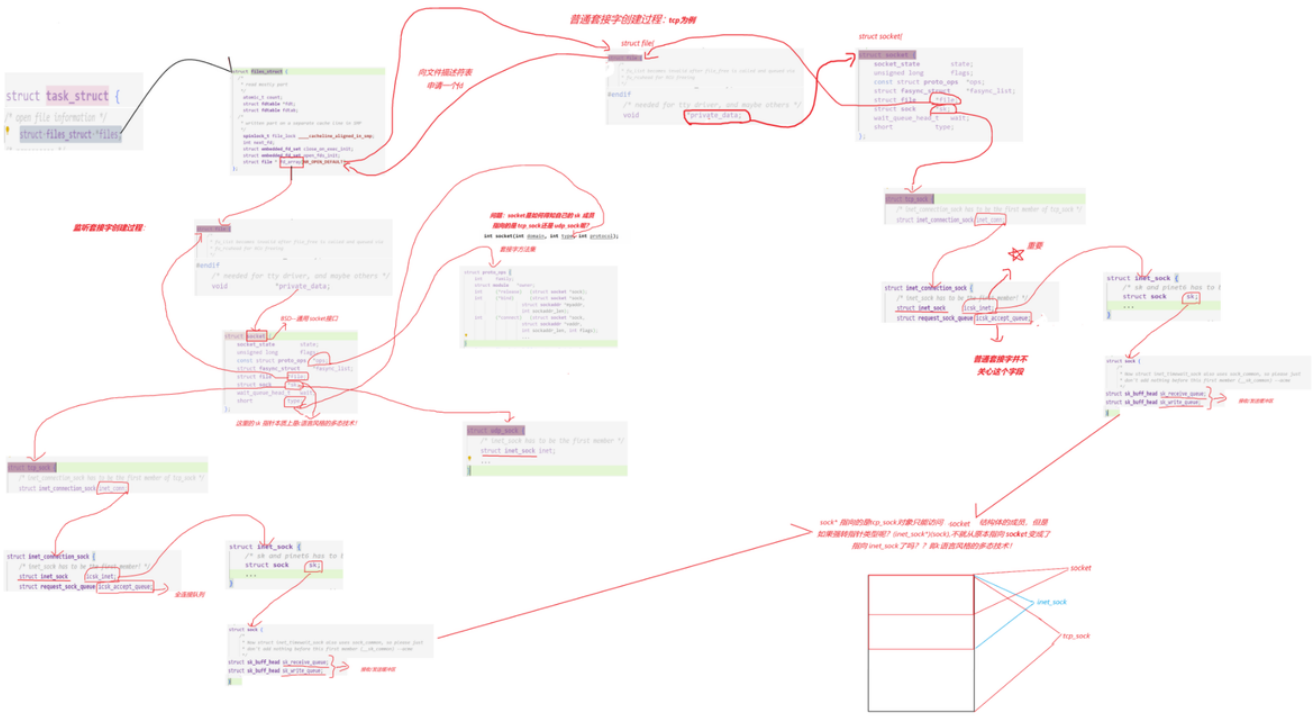
全连接队列

```
struct inet_sock {
    /* sk and pinet6 has to b
    struct sock     sk;
    ...
}
```

```
struct sock {
    /*
     * Now struct inet_timewait_sock also uses sock_common, so please just
     * don't add nothing before this first member (__sk_common) --acme
     */
struct sk_buff_head sk_receive_queue;
struct sk_buff_head sk_write_queue;
}
```

接收/发送缓冲区

sock* 指向的是tcp_sock对象只能访问struct sock结构体的成员，但是如果强转指针类型呢？(inet_sock*)(sock),不就从原本指向sock，变成了指向inet_sock了吗？？即c语言风格的多态技术！



创建普通socket连接：